

Buildings on the map, from LiDAR + GIS

Extracting building heights and volumes, rendered to true scale on the minimap.

GISViz · implementation plan · 3 August 2026 · rev. 2 (EXIF GPS georegistration)

What already exists

Most of the ingredients are already in the repo. Building height is just `DSM - DEM`, and `process_lidar.py` can already produce both from a single `.laz` tile.

Piece	Where	Status
Ground DEM (class-2 returns, MIN z)	<code>process_lidar.py --kind dem</code>	works
Surface DSM (all returns, MAX z)	<code>process_lidar.py --kind dsm</code>	works
Building footprints	<code>process_vectors.py</code> → <code>*_buildings_4326.geojson</code>	works
Footprints in PostGIS	<code>osm.ways</code> (polygon + tags JSONB)	works
Layer registry	<code>raster_layers</code> , <code>vector_layers</code>	works
Minimap renderer	<code>OSMViewer.jsx</code> (hand-rolled SVG)	blocked

Four problems to resolve first

- 1 The minimap is currently dead.** `OSMViewer.jsx:198` calls `parseOsm("")` on an empty string at module load. That throws, is caught, and the component renders “Building the map square...” indefinitely. Nothing will appear on the map until it has a real data source. This is step zero.
- 2 Nothing is in a metric CRS.** The pipeline reprojects everything to EPSG:4326 by design. Area and volume computed in degrees are meaningless, and degrees-per-metre varies with latitude. All zonal maths must happen in the LiDAR’s *native* UTM CRS (e.g. EPSG:6347), where the cell size is already in metres.
- 3 The minimap projection is not to scale.** `makeProjector` (`OSMViewer.jsx:121`) maps the latitude and longitude spans onto the same pixel extent with no `cos(latitude)` correction, stretching everything horizontally — by roughly 40% at 45°N. This contradicts the “actually scaled” requirement and needs fixing regardless of the rest.
- 4 Splat scale is arbitrary.** COLMAP reconstructions carry no metric scale unless given GPS or ground control points. *If uploaded photos carry EXIF GPS, this is largely solved* — see Phase 4. Either way it does not block Phases 1–3.

Phase 1 — Heights and volumes

Run `process_lidar.py` twice per tile at the **same** `--cell`, once `--kind dem` and once `--kind dsm`. Both grids derive from the LAS header bounds and cell size, so they come out pixel-aligned and can be subtracted directly.

New module: `src/building_heights.py`

Per footprint, reprojected from 4326 into the LiDAR's native CRS:

```
ground = percentile(DEM cells in a ~2 m ring outside the footprint, 10)
roof   = percentile(DSM cells inside the footprint, 75)
height = max(roof - ground, 0)

volume_lidar =  $\sum \max(\text{DSM} - \text{ground}, 0) \times \text{cell}^2$   # true volume under the roof
volume_prism = area  $\times$  height  # what a flat extrusion shows
coverage    = valid cells  $\div$  total cells
```

Percentiles rather than min/max matter: chimneys, antennas and parapets wreck a MAX, and a single stray low return wrecks a MIN.

Store *both* volumes. They agree within ~10% on flat roofs and diverge on pitched ones, which is a useful correctness signal in itself.

Keep `coverage` too. Footprints outside the tile or under dense canopy will have no usable data, and those should render differently rather than as a confidently wrong 0 m building.

No new dependencies. `rasterio.features.geometry_mask` plus numpy covers this, and both are already in use. `rasterstats` would be marginally less code if an extra dependency is acceptable.

Phase 2 — Storage and API

New table `public.buildings`:

- `geom` GEOMETRY(Polygon, 4326), `osm_id`, GIST index on `geom`
- `ground_m`, `roof_m`, `height_m`, `footprint_area_m2`
- `volume_prism_m3`, `volume_lidar_m3`, `coverage`
- `lidar_layer_id`, `job_id`, `created_at`

Geometry stays in 4326 for the map; every measurement is a metric scalar computed in UTM.

`GET /gis/buildings?bbox=...` returns a GeoJSON FeatureCollection with height, area and volume in the feature properties. This fits the existing `/gis/*` surface in `gis_api.py`.

Phase 3 — Minimap

Keep the hand-rolled SVG. It is a smaller change than adopting a map library, and 2.5D extrusion in SVG is straightforward:

1. Fix `makeProjector` with a `cos(midLat)` factor on the longitude axis so the scale is true.
2. Per building, draw the footprint, the roof polygon translated up by `k × height_m`, and wall quads joining corresponding edges.
3. Apply a painter's algorithm — sort by latitude and draw far to near, or buildings will overlap incorrectly.
4. Fill by volume using a sequential palette, with a legend and a metric scale bar.

Buildings with low `coverage` get a flat hatched footprint, visually distinct from a measured one.

Alternative: MapLibre GL's `fill-extrusion` gives real 3D, tilt and correct depth sorting for free, at the cost of a dependency and rewriting `OSMViewer.jsx`. Worth it if the minimap should pan, zoom and tilt; overkill if it stays a static inset.

Phase 4 — Buildings in the 3D viewer

4a. The building mesh

`THREE.Shape` + `ExtrudeGeometry` per footprint, built in a local ENU frame centred on the scene origin, at **1 three.js unit = 1 metre**. That yields a metrically correct building mesh that can be measured against.

4b. Georegistering the splat

Placing that mesh in the same space as a splat needs a 7-DOF similarity transform. The splat currently sits at an arbitrary `scale 0.9` in `SplatViewer.jsx` because COLMAP output has arbitrary scale, position and orientation.

If uploaded photos carry EXIF GPS, COLMAP solves this directly. Both halves are already present in the 3.8 build the Modal image compiles:

- `feature_extractor` parses EXIF GPS into per-image priors (`image_reader.cc` writes lat/lon/alt into `TvecPrior`), so the data reaches the COLMAP database with no extra work.
- `model_aligner` consumes those priors and solves the transform: `--ref_is_gps 1`, `--alignment_type enu`, `--estimate_scale 1`, `--robust_alignment 1` (RANSAC), `--min_common_images 3`, and `--transform_path` to write the resulting Sim3 out for storage.

The `enu` alignment type produces exactly the local East-North-Up metric frame 4a wants — gravity-aligned, north-oriented, 1 unit = 1 m. It replaces the hand-tuned `rotation.set(Math.PI, 0.25, 0)` entirely.

Accuracy ceiling

Phone GPS is 3–10 m horizontally, and worse in urban canyons from multipath — exactly where buildings get photographed. **Absolute placement therefore stays sloppy at metre scale.**

Scale comes out far better than that implies. It is estimated from the baseline across all cameras rather than any single fix, so errors average down under RANSAC. Photos spread over tens of metres with ± 5 m per-camera noise typically yield scale good to a few percent — and scale is the number that matters here. More cameras with valid GPS helps materially; the default `--min_common_images 3` is a floor, not a target.

The vertical trap. EXIF `GPSAltitude` is usually WGS84 *ellipsoidal* height, while LiDAR DEMs are *orthometric* (NAVD88 or similar, geoid-referenced). The geoid undulation between them is tens of metres and varies regionally — roughly –30 m across much of the US. Taking altitude from EXIF and ground elevation from the DEM produces a large constant offset that looks like a bug but is not.

Use GPS for horizontal position and scale only, and sample ground elevation from your own DEM at the resulting lat/lon. The LiDAR ground surface is far more accurate vertically than any phone GPS, so this is both simpler and more correct.

What EXIF GPS does not fix

- **Building heights.** Those come from LiDAR in Phase 1, which beats anything photo-derived. GPS places the splat; it does not measure it.
- **Splat volume.** A Gaussian splat is not a watertight mesh; volume still comes from the LiDAR zonal statistics.

- **Missing metadata.** Screenshots, crops, most editing tools and some upload paths strip EXIF. Surface “N of M photos had GPS” in the job status rather than silently falling back to an unscaled model.

Integration caveat

Do not assume EXIF survives into the images COLMAP reads. `ns-process-data` copies and downscales into `processed/images` , and that re-encode may drop GPS. Read the tags from the *original* downloaded photos in the Modal worker — they are already on disk at `images_dir` before processing — write a `ref_images.txt` , and pass it to `model_aligner` explicitly rather than trusting the database priors.

Persist the Sim3 from `--transform_path` alongside the splat so the frontend knows where, and at what scale, to place it.

Decisions worth making now

Decision	Trade-off
Footprint source	GeoJSON from <code>process_vectors.py</code> , or query <code>osm.ways</code> directly? The DB path avoids a file round-trip and gives bbox queries for free.
When heights are computed	A new step in <code>gis_worker.py</code> when a LiDAR job and an OSM job overlap, or a separate on-demand endpoint? The former is automatic but couples two job types.
Minimap renderer	SVG 2.5D (recommended) vs. MapLibre <code>fill-extrusion</code> .
EXIF GPS handling	Require it, or treat it as an optional upgrade? Requiring it makes every splat metrically placeable but rejects screenshots and stripped uploads. Optional means carrying two code paths — aligned and unaligned — through the viewer.

Logistics: this work spans three branches. The GIS pipeline is on `backend` , the viewer is on `frontend_new_renderer` , and that branch carries a third, older backend at `src/server/main.py` with no `/jobs` or `/gis` routes. Worth reconciling before starting, or Phase 2’s API will land somewhere the frontend cannot reach it.